

ghostly-runs: Optimizing and Benchmarking ghostly-neighbors for Neighborlist Construction

Michael Almeida

mjalmeida172@gmail.com

5/7/2026

cs2050

Introduction

Molecular dynamics (MD) simulations advance the positions of atoms under different conditions. Many MD frameworks contain a subroutine called **neighbor list construction**: given N atoms and a cutoff radius r_c , find every pair (i, j) with distance $d(i, j) < r_c$. The naïve all-pairs approach is $O(N^2)$, but a common optimization is to use a "cell-list". This reduces the complexity to $O(N)$ by partitioning space into cells of side length $\geq r_c$, and only checking the 27 neighboring cells for each atom.

Despite the algorithmic simplicity, achieving high throughput is non-trivial. Every decision, such as cache-friendly memory layout, auto-vectorization (or lack thereof), thread-level parallelism, and GPU execution, needs to be scrutinized. In this project, we take the algorithm and walk through 5 parallelization regimes: serial C++ with hand-rolled AVX-512 intrinsics, OpenMP shared-memory threading, MPI distributed-memory decomposition, CUDA, and ultimately "productivity languages" — Numba, JAX, and Kokkos. The serial, OpenMP, CUDA, and kokkos productivity-framework backends were authored for this report; the MPI domain decomposition used for benchmarking was provided in the ghostly-neighbors package. All backends are verified against a common reference and benchmarked on the Harvard HUIT cluster.

Methods

Algorithm Overview

The core algorithm proceeds in four stages:

1. **Cell assignment.** Each atom is mapped to a 3D grid cell of side length r_c .
2. **Cell-list construction.** A multi-pass algorithm is used: histogram, scan, scatter. First, cell-atom counts are computed (histogram). Second, a prefix sum is used to create offsets (scan); finally, atoms are allocated to their respective cells (scatter). This is also known as a 'counting sort'.
3. **Pair search.** For each owned atom i , iterate over atoms in the 27 neighboring cells (the $3 \times 3 \times 3$ stencil). Compute squared distances and accept pairs below r_c^2 .
4. **Output.** Store the index pair (i, j) and displacement vector d_{ij} for each accepted pair.

Part 1: Serial Implementation

The serial baseline comes in two flavors:

- **cpp_serial_scalar**: A plain $O(N^2)$ loop without cell lists. This is brute force version of the algorithm.

- **cpp_serial_avx512**: The same $O(N^2)$ loop but with AVX-512 intrinsics processing 8 atom pairs per SIMD lane.

Part 2: OpenMP Implementation

The OpenMP version (**cpp_binned_avx512_two_pass**) has four phases: cell list build, pair count, scan, scatter. The cell list building itself uses the same pattern: histogram, scan, scatter; highlighting the usefulness of this pattern in minimizing data movement and maximizing FLOPs.

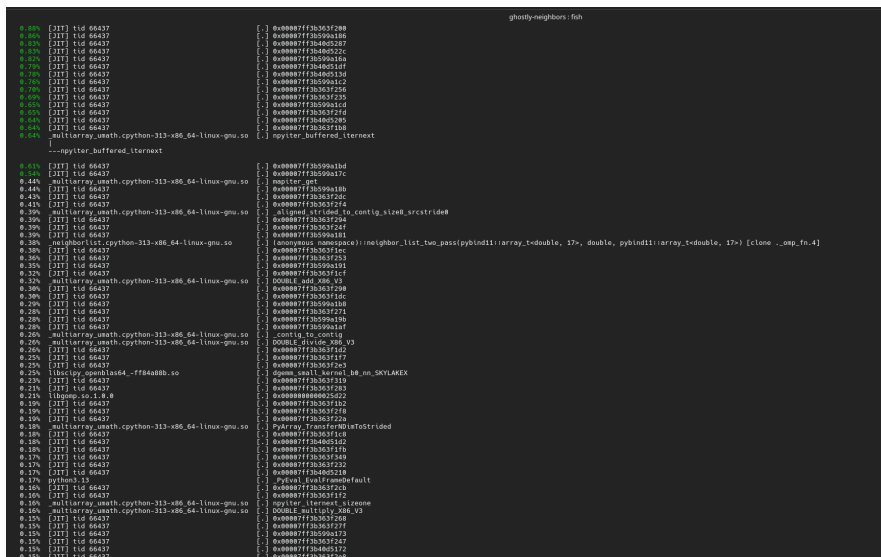
The algorithm is parallelized in different areas.

- **Cell-list build**: The SoA coordinate copy is threaded using a `#pragma omp parallel for` as there are no races. Min/max reduction to compute the cell is done using a `#pragma omp parallel for reduction(...)` over to compute the stencil offsets. There is an additional reduction over calculating the maximum number of atoms in the collective cells.
- **Pass 1 (count)**: `#pragma omp parallel for schedule(static)` over the *i* atoms (owned atoms).
- **Prefix sum**: Sequential $O(N)$ scan to compute output offsets. Herein lies another great optimization opportunity (a parallel scan via `#pragma omp parallel for simd reduction(incscan, +=...)` or a Kogge-Stone or Brent-Kung) that is left as a TODO.
- **Pass 2 (lock free write)**: parallel decomposition in a `#pragma omp parallel` block. Each thread writes to a disjoint slice of the output arrays determined by the prefix sum, so no locks are needed.

C++ Profiling with perf

*Figure 4a: `perf report` for the

single-pass fused kernel. 53.7% of CPU cycles sit inside `neighbor\_list\_serial`, with the call tree dominated by `std::vector::push\_back` reallocation paths (~35% `push\_back`, ~7% `emplace\_back`,



*Figure

4b: `perf report` for the full Lennard-Jones simulation using the two-pass kernel. The neighbor list kernel (`neighbor\_list\_two\_pass`) now accounts for just **0.38%** of total cycles.

Before arriving at the two-pass design, **perf record** + **perf report** on the single-pass fused kernel revealed that the majority of runtime was not spent on distance computation but on **dynamic memory allocation**. This of course assumes a certain problem size — for the LJ simulation we profiled, this was 108,000 Cu/FCC atoms (30^3 repeat) at a 12 Å cutoff. The call tree showed **std::vector::push_back** and its internal reallocation path consuming roughly 50% of total cycles. Each time a thread triggered its vector's allocation capacity, it would have to reallocate. This, at times, meant shuffling around gigabytes of atom data: at 30 repeat / 12 Å, the kernel produces ~68.8M pairs (1,274 neighbors per atom × 108k / 2), each stored as a 32-byte tuple (i, j, dx, dy, dz) for ~2.2 GB of final storage. With **std::vector**'s doubling realloc strategy, every doubling moves the entire current contents — summed across the resize chain, each thread shuffles roughly 2× its share, for ~4.4 GB of memory moved per neighbor-list construction. And this happens every MD step.

This profiling directly motivated the two-pass design. At first, it seems wasteful to expend an entire pass just computing something so meaningless as the count of pairs. This computation, however, lets us directly allocate vectors of proper sizes so we never have to pay crazy realloc costs. Alas, this also means that at smaller problem sizes, the single-pass kernel is expected to be faster than the multipass.

The before/after contrast is stark: the neighbor list went from consuming **53.7%** of application cycles to **0.38%** — a ~140× reduction in its share of runtime. It made the kernel fast enough that the entire force computation now dominates the profile. This is the ideal outcome of optimization: the target kernel vanishes into the noise.

Part 3: MPI Decomposition (provided) and benchmarking

ASIDE The MPI domain decomposition that is being benchmarked is *not implemented by me*. The **DistributedParticles** and **DistributedParticlesMorton** classes — including the Cartesian topology setup, halo exchange, ghost-position refresh, and force accumulation — were already provided as part of the project scaffold. My contribution on the MPI side was to build the benchmarking harness around these classes, exercise them across the full atom-count × cutoff × rank-count grid, write the SLURM infrastructure to drive the benchmarking, and analyze the resulting performance and failure modes. The description below documents what the provided implementation does so the benchmarks downstream are interpretable; it is not a description of

code I authored. Although I did author a simplified 2D Slab decomposition in C++ to meet project requirements, it is left unbenchmarked in this report.

'ghostly-neighbors' uses MPI to decompose the simulation across different ranks. The neighborlist is then run on each rank's locally-owned atoms, with a ghost layer of thickness r_c providing the boundary information needed for cross-rank pairs. Two decomposition strategies sit behind a common `DistributedParticles` interface that was already provided.

- **Cartesian decomposition:** `MPI.Compute_dims(comm.size, 3)` auto-balances the rank count into a Cartesian grid, which `comm.Create_cart(dims, periods=[True, True, True])` wraps into a periodic 3D Cartesian communicator. Each rank owns its own axis of the simulated cell.
- **Morton-ordered decomposition:** A `DistributedParticlesMorton` strategy assigns atoms to ranks along a space-filling Z-order curve. Only Cartesian is exercised in the benchmark sweeps in this report.

(aside) Cluster setup and SLURM complexity

Running this code on the HUIT cluster was tough. The benchmarking harnesses and slurm scripts are all contained inside `ghostly-runs`.

Some footguns:

- **Two separate Python virtualenvs.** CPU-only and GPU benchmarks use different virtual environments: `ghostly-neighbors/.venv` and `ghostly-neighbors-gpu/.venv` respectively. The hand-rolled intrinsics were built on a compute node, and the GPU nodes do not have support for the intrinsics, which caused SIGILLs.
- **GPU rank-to-device pinning is manual.** `srun --gpu-bind=closest` interacts badly with `mpirun` underneath. The fix was a tiny inline wrapper (`.gpu_bind.sh`) that sets `CUDA_VISIBLE_DEVICES=$OMPI_COMM_WORLD_LOCAL_RANK` before running the Python interpreter, ensuring each rank sees exactly one GPU.
- **GRES quirks.** The GPU partition requires `--gres=gpu:N`, which on a 4xL4 node means we couldn't run the 1- and 2-rank cases in a smaller allocation — the whole node had to be reserved up-front. In general, allocating GPUs was a bit tricky and hacky, and this ultimately led to spiky/weird behavior in our benchmarks as the GPUs were contested, and exclusive allocations are prohibited.

Part 4: CUDA Implementation

The GPU version maps the C++ neighbor search to CUDA threads, compiled at runtime via CuPy RawModule. All benchmarking was done on GPU nodes allocated from the HUIT cluster, each of which contains 4 NVIDIA L4 GPUs.

The development of the kernels was performed on an NVIDIA GeForce RTX 5090. The process iteratively optimized the kernel through three stages, ultimately resulting in three different kernels, each guided by Nsight Compute profiling. There was no code churn between the two architectures; the build system and supporting infrastructure automatically determine the right architecture to compile for.

On both the 5090 and the HUIT Cluster's L4s, FP64 is nerfed to 1/64 of FP32 — ~473 GFLOPS FP64, ~300 GB/s BW.

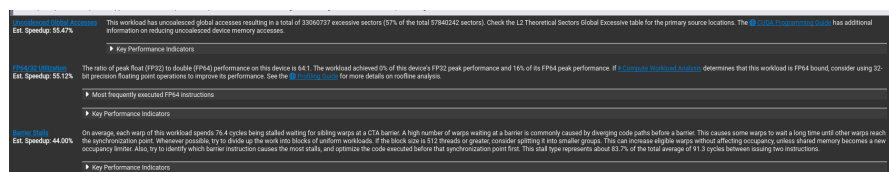
Kernel 1 — Naive (one thread per atom). Each CUDA thread handles one owned atom i , iterating over the 27 neighboring cells in global memory. A two-pass strategy (count then write) mirrors the CPU design. Block size 256, grid sized to cover all owned atoms. Nsight Compute flagged this as **latency-bound**: SM throughput 53.7%, IPC 0.42, with 57% excessive (uncoalesced) global memory sectors and barrier stalls consuming 84% of warp cycles.

Kernel 2 — Tiled with shared memory. Cell-list segments for neighboring cells are loaded into shared memory tiles before the distance-check loop, improving spatial locality. The tile dimensions were naively set to match the block size (256). The motivation for this optimization was to improve on uncoalesced memory accesses — and it did, raising memory throughput to 23.8% and reducing uncoalesced sectors to 35%... but barrier stalls remained dominant (84% of cycles) as warps waited for each other at `__syncthreads()` boundaries. Moreover, there was an additional synchronization step in the CuPy kernel launch code that was eliminated, as there was a misunderstanding on how CUB should be invoked from CuPy and instead the prefix scan was being done via NumPy.

Kernel 3 — Warp-level primitives. Along with unrelated CuPy kernel fixes, the final kernel targets the barrier-synchronization overhead. In the tiled version, there were bottlenecks at `__syncthreads()` as each cell list would contain a different amount of work, and at the edges (cell lists don't necessarily come in packages of 32, which is the number of threads per warp) we were incurring great overhead waiting. One approach was to pad the cell lists to offsets of 32 and use a self-masking procedure akin to the C++ pass. This code relied heavily on different synchronization points and was incredibly complex. A second approach, as an alternative to shared memory tiling, was to guard the global output array in a mutex — but with so many threads fighting for the lock this certainly would not have fixed the problem. Ultimately, we opted to use a special form of the global mutex, in which the kernel can effectively pipeline many writes per warp into the shared output array via SM intrinsics — called a ballot/shuffle. This replaces block-level synchronization with warp-level shuffles and vote intrinsics, eliminating shared memory barriers entirely for the inner distance-check loop. Grid size (37,596, 1, 1) × block (128, 1, 1) with 40 registers per thread. This achieved **84.3% SM throughput**, IPC 0.82, and reduced uncoalesced sectors to 19%. Nsight Compute reclassified the kernel from "Latency Issue" to "**High Throughput**", with the FP64 pipeline identified as the primary bottleneck at 84% utilization — meaning the kernel is now compute-bound on the FP64 unit rather than stalled on memory or synchronization.

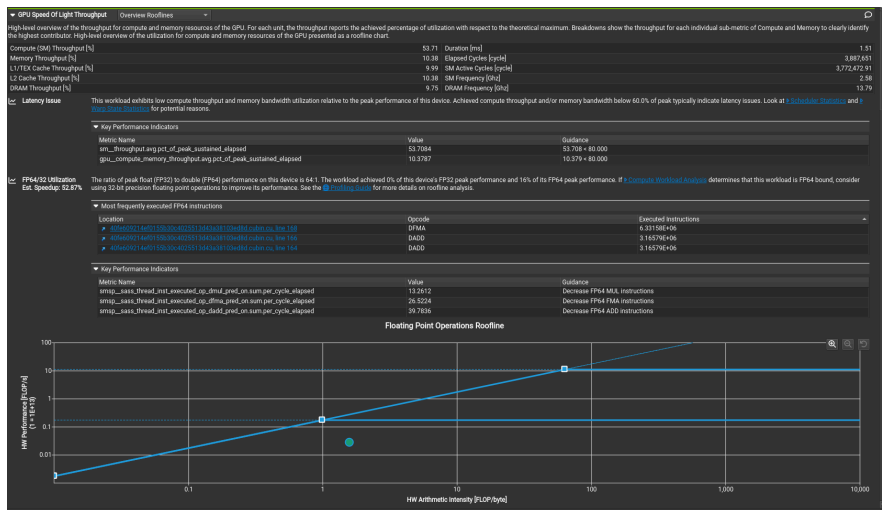
The ballot/shuffle pattern walkthrough is included in the appendix.

Stage 1: Naive Kernel



*Figure 7a: Nsight Compute

diagnostics for the naive ``get_pairs`` kernel. Uncoalesced global accesses (57% excessive sectors, est. 55% speedup if fixed), FP64/32 utilization warning (64:1 ratio, achieving 16% of FP64 peak), and barrier stalls consuming 83.7% of warp cycles.*

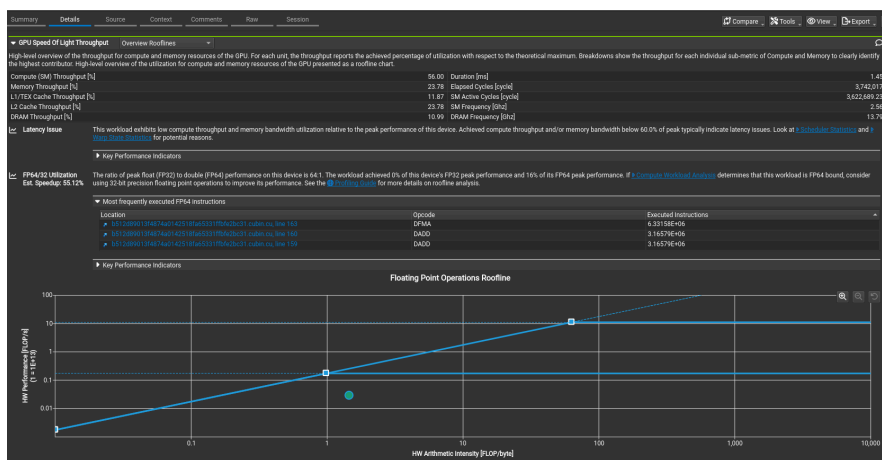


*Figure 7b: Nsight Compute Speed

of Light and roofline for the naive kernel. SM throughput 53.7%, memory throughput 10.4%. The kernel (green dot) sits well below the bandwidth ceiling at $AI \approx 1$ FLOP/byte. Nsight flags a "Latency Issue" — both compute and memory utilization are below 60%.*

The naive kernel launches one thread per owned atom with block size 256. Nsight Compute identified three major issues: uncoalesced global memory accesses from the irregular cell-list gather pattern (57% of sectors wasted), barrier stalls where warps waited an average of 76.4 cycles at CTA barriers (83.7% of all stall cycles), and low IPC (0.42) indicating the pipeline was mostly idle.

Stage 2: Tiled with Shared Memory

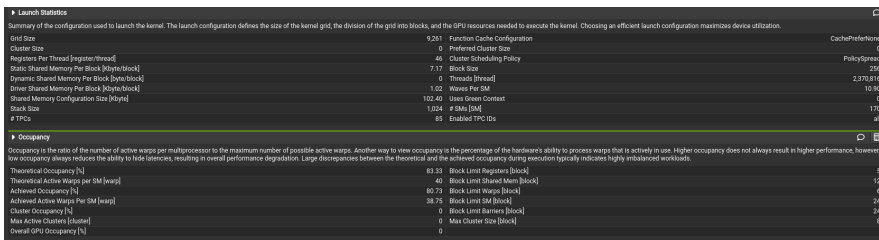


*Figure 7c: Nsight Compute for the

tiled kernel. SM throughput improved to 56%, memory throughput to 23.8%. Barrier stalls still dominate (84% of cycles), but uncoalesced sectors dropped to 35%. FP64/32 utilization est. speedup 55%.*

Loading cell-list segments into shared memory tiles before the distance-check loop improved memory throughput from 10.4% to 23.8% and cut uncoalesced sectors from 57% to 35%. However, barrier stalls remained the dominant bottleneck — the `__syncthreads()` calls required for tile loading introduced synchronization points where fast warps idled waiting for slow ones.

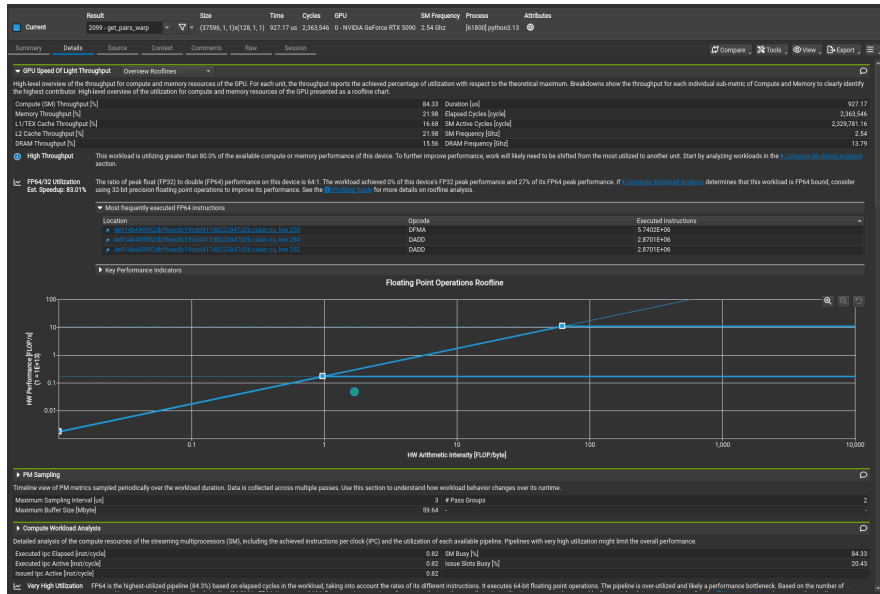
Stage 3: Warp-Level Kernel



*Figure 7d: Launch configuration for

the warp kernel. Grid 9,261 blocks $\times$ 256 threads, 46 registers/thread, 7.17 KB shared memory/block.

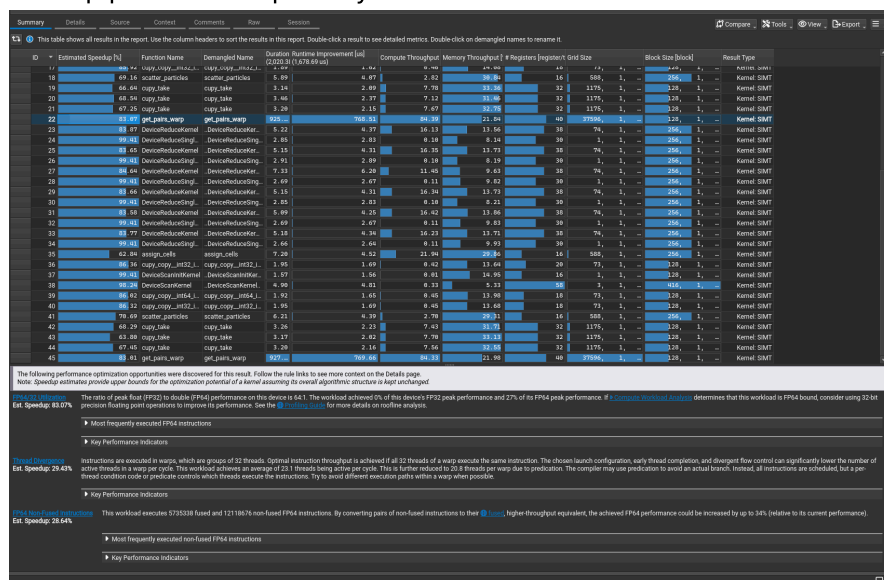
Achieved occupancy 80.73% (theoretical 83.33%), with register count as the limiting factor (5



*Figure 7e: Nsight

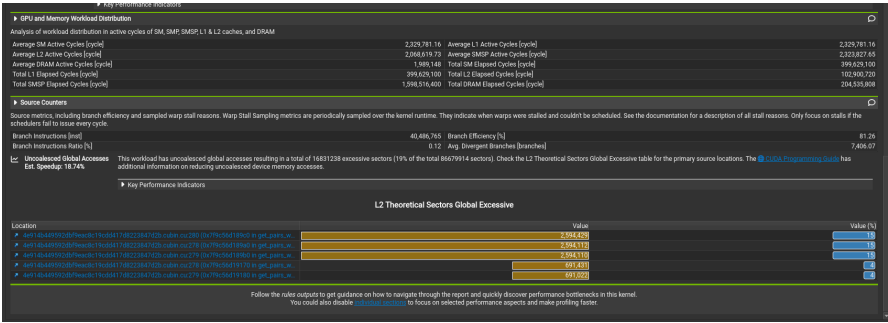
blocks/SM).*

Compute for the final `get\_pairs\_warp` kernel. SM throughput **84.3%**, memory throughput 22.0%, IPC 0.82. Nsight reclassifies this as **"High Throughput"** — utilizing >80% of available compute. The FP64 pipeline is the primary bottleneck at 84% utilization.*



*Figure 7f: Nsight Compute

summary view showing all kernel launches. ``get_pairs_warp`` (row 22/45) dominates at 925 μ s with 84.39% compute throughput. Nsight identifies thread divergence (est. 29% speedup) and non-fused FP64 instructions (est. 29% speedup) as remaining optimization opportunities.*



*Figure 7g: Memory workload and

source counters for the warp kernel. Branch efficiency 81.26% with 7,406 divergent branches. Uncoalesced global accesses down to 19% excessive sectors — the top L2 access hotspots correspond to the coordinate load instructions in the distance-check loop.*

The three kernel variants show a clear optimization trajectory:

Metric	Naive	Tiled	Warp
SM Throughput	53.7%	56.0%	84.3%
Memory Throughput	10.4%	23.8%	22.0%
IPC	0.42	—	0.82
Uncoalesced excessive sectors	57%	35%	19%
Top bottleneck	Latency (barrier stalls)	Latency (barrier stalls)	FP64 pipeline (84%)
Nsight classification	Latency Issue	Latency Issue	High Throughput

The dominant optimization was eliminating barrier stalls. In the naive and tiled kernels, warps spent ~84% of cycles stalled at CTA (Cooperative Thread Array) barriers waiting for sibling warps. The warp-level kernel removes this synchronization overhead entirely by operating within a single warp (32 threads) using shuffle instructions.

The remaining uncoalesced accesses (19%) stem from the irregular nature of the cell-list gather: atoms in neighboring cells are not contiguous in memory, so coordinate loads produce scattered access patterns. This is a fundamental characteristic of the data structure rather than a fixable inefficiency. This potentially could have been optimized away if the atom pairs were passed in Morton/Z order, as they would have been sorted. Alternatively, a sorting pass could have been done after the initial prefix scan, which was not attempted due to correctness concerns in a GPU context.

Nsight Compute's suggestion to switch to FP32 (estimated 83% speedup) aligns with the roofline — the kernel now saturates the FP64 pipeline, and the FP64:FP32 ratio of 1:64 on this device means enormous headroom exists if precision requirements were relaxed or NVIDIA stopped nerfing us.

Part 5: Additional Implementations

Three additional frameworks are compared:

- **Numba** (numba): JIT-compiled Python using @njit with cell-list logic. Provides a "pure Python" baseline with automatic SIMD and loop optimization via LLVM. This implementation was not

authored by me.

- **Kokkos** (`kokkos_cpu` and `kokkos_gpu`): Performance-portable C++ library using `Kokkos::parallel_for` with a `RangePolicy`. Built with the OpenMP backend on CPU. Demonstrates the abstraction overhead of a portable framework. Unfortunately I did not include this-- see additional notes in Framework Comparison for details.
- **JAX** (`jax`): A `jax.jit`-compiled neighbor list using `jnp` array operations and `lax.fori_loop` over cell stencils. Demonstrates the "compiled-Python" end of the productivity spectrum, but is restricted to single-GPU runs (see the Framework Comparison for why JAX is excluded from the MPI sweep). This was not authored by me.

These serve as a comparison axis for programmer productivity vs. raw performance.

Results

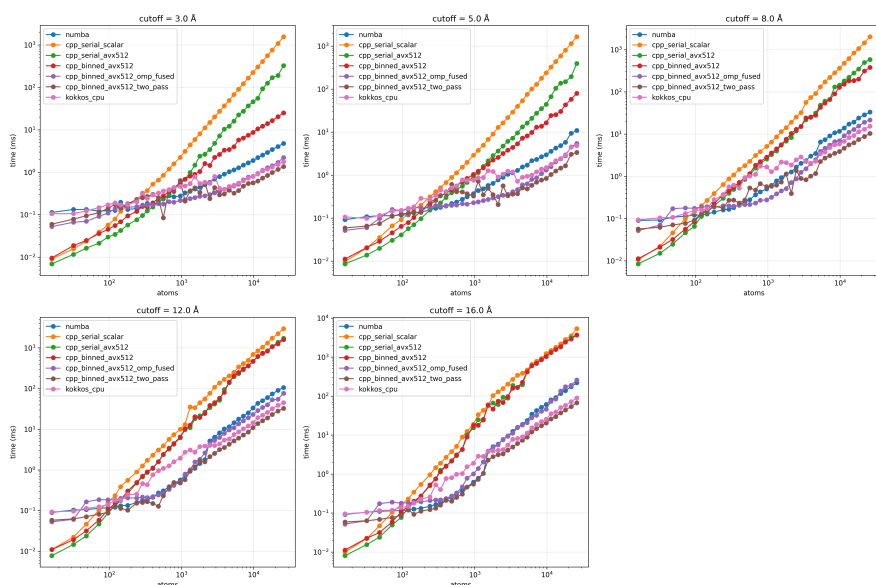
Correctness Verification

All 8 backends (`cpp_serial_scalar`, `cpp_serial_avx512`, `cpp_binned_avx512`, `cpp_binned_avx512_omp_fused`, `cpp_binned_avx512_two_pass`, `numba`, `kokkos_cpu`, `jax`) were verified against each other — and against ASE's `neighborlist` for systems ≤ 1000 atoms — across 73 test configurations spanning:

- Three crystal structures (C/diamond, Cu/fcc, Fe/bcc) at multiple supercell sizes
- Four cutoff radii (3.0, 5.0, 8.0, 12.0 Å)
- Edge cases: ghost atoms, random configurations, single-atom systems, zero-pair cutoffs, and single-cell-exceeds-box scenarios

All 73 cases passed across all backends. The associated slurm script is called `correctness.slurm`. The validation results can be found in `ghostly-runs/runs/correctness/ghostly-correctness-*.out`

CPU Benchmark: All Backends



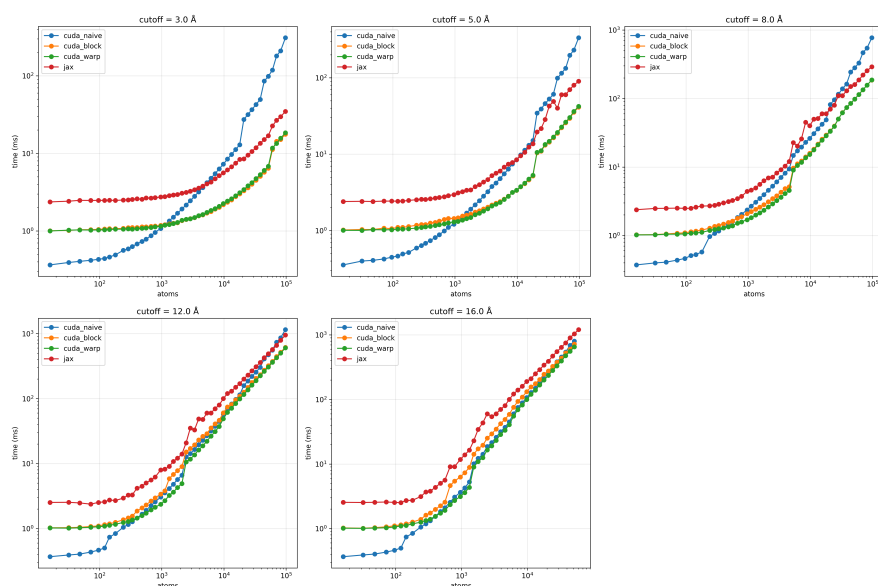
*Figure 1a: Runtime vs. atom count

for all CPU backends at five cutoff radii (log-log scale). The naïve $O(N^2)$ kernels (`cpp_serial_scalar`, `cpp_serial_avx512`) diverge from the $O(N)$ cell-list kernels beyond ~ 1000 atoms. Kokkos shows high constant overhead (~ 7 ms) from its runtime initialization but matches cell-list scaling at large N .*

Several trends are visible:

- **AVX-512 vs. scalar:** The SIMD serial kernel is consistently $\sim 2\times$ faster than the scalar baseline.
- **Cell-list crossover:** The cell-list variants win decisively beyond ~ 1000 atoms.
- **Cutoff sensitivity:** At larger cutoffs the neighbor density increases, expanding the number of pairs per cell-stencil evaluation. The cell-list kernels scale gracefully; the $O(N^2)$ kernels are cutoff-insensitive since they check all pairs regardless.
- **Kokkos overhead:** The `kokkos_cpu` backend shows $\sim 5\text{--}7$ ms of constant initialization cost that dominates at small system sizes. At large N ($>10k$ atoms) it converges toward the hand-written cell-list performance.
- **Numba:** Competitive with the C++ cell-list at small N but falls behind at large N , likely due to less aggressive SIMD utilization compared to explicit AVX-512 intrinsics.

GPU Benchmark: All Backends



*Figure 1b: Runtime vs. atom count

for all single-GPU backends at five cutoff radii on the L4 (log-log scale). `kokkos_gpu` is omitted from this plot — see the Framework Comparison section for the full story.*

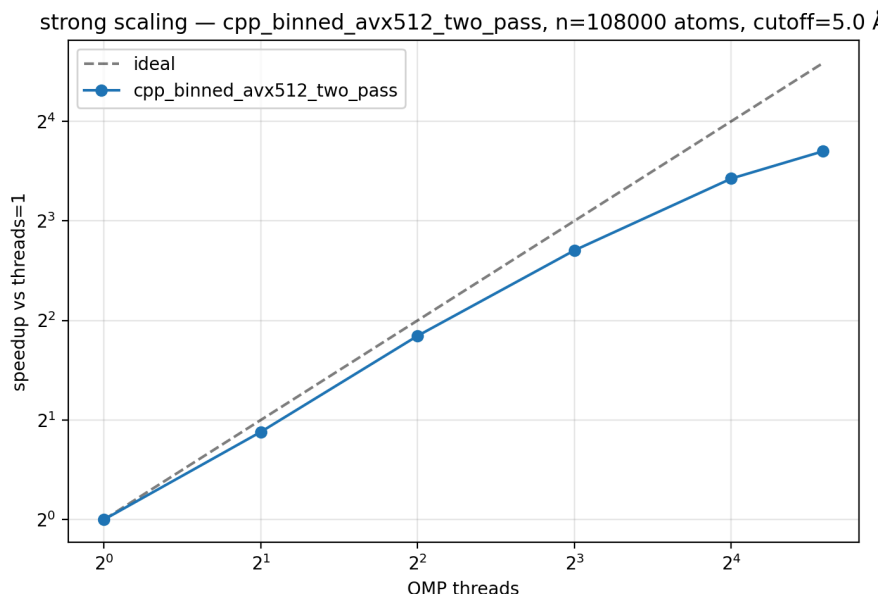
The GPU side recapitulates the same optimization journey as the CPU. Where we eliminated allocator overhead via prefix-summed two-pass writes, the GPU optimization eliminated global-stall latency via warp write intrinsics and shuffle atomics. Not to mention optimizing to leave as much memory on-device as possible to minimize data shuttling.

A few specifics are visible in the curves:

- **cuda_block and cuda_warp track each other closely** through most of the sweep — the warp kernel's elimination of `__syncthreads()` matters more for small problems where stalls dominate, and at the largest sizes both are saturating the same FP64 pipeline (see Figure 5 roofline).
- **cuda_naive is the slowest CUDA variant** at small/medium N (latency-bound, as Nsight Compute confirmed) but catches up at large N where there's enough work to hide its uncoalesced gathers.
- **JAX is competitive at small N** (its constant XLA dispatch cost amortizes well below $\sim 1k$ atoms) but falls behind at scale because the high-level `jit`-compiled implementation produces a less specialized kernel than the hand-written CUDA versions.
- **No kokkos_gpu curve** appears here despite Kokkos being one of the project's headline portability stories. Due to what is most likely an implementation bug on my end, Kokkos would OOM pretty

early on due to preallocating its Views. This OOM would then cause a global deadlock in the **Allreduce** harness; it was simpler to simply move past this. On the MPI variant, however, the memory footprint is much smaller and this preallocation bug was not an issue.

OpenMP Strong Scaling



*Figure 2: Strong scaling of

cpp_binned_avx512_two_pass on 108,000 atoms, cutoff = 5.0 Å. Dashed line shows ideal linear speedup.*

Strong scaling results for 108,000 Cu/FCC atoms at 5.0 Å cutoff show **12.98× speedup at 24 threads** (54% parallel efficiency) on the Xeon 8275CL, with the runtime dropping from 201.7 ms at 1 thread to 15.5 ms at 24. The initial parallelization of the cell-list build phase was critical — without it, Amdahl's law limited total speedup to ~1.13× regardless of thread count due to the serial cell construction consuming over 60% of the single-threaded runtime.

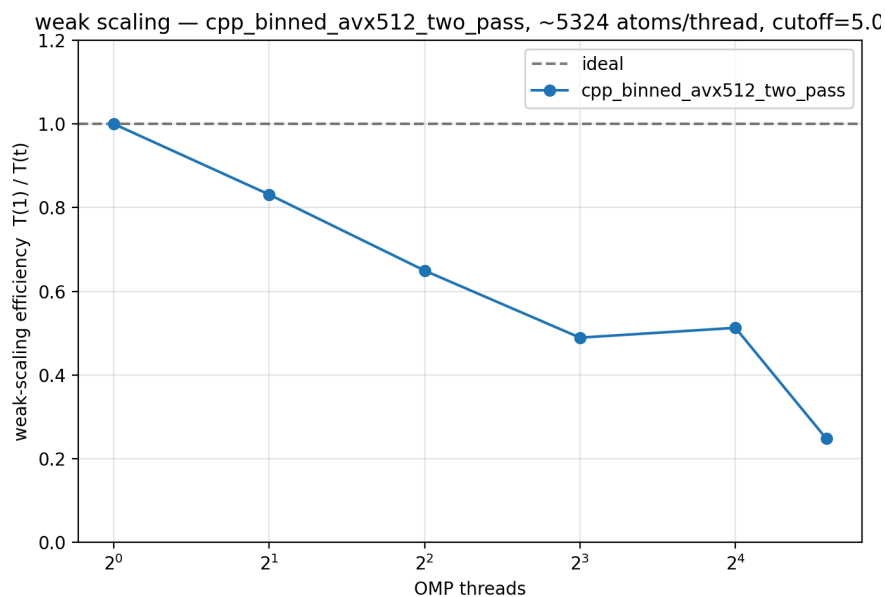
Threads	Runtime (ms)	Speedup	Efficiency
1	201.7	1.00×	100%
2	109.6	1.84×	92%
4	56.1	3.59×	90%
8	31.0	6.51×	81%
16	18.8	10.74×	67%
24	15.5	12.98×	54%

Key observations:

- **Memory bandwidth:** Beyond 16 threads, efficiency drops noticeably (67% → 54% from 16 to 24 threads) as the AVX-512 inner loop and the cell-list gather start saturating the 141 GB/s DRAM bandwidth on the single socket.
- **Hyperthreading effects:** The Xeon 8275CL has 24 physical cores with 2-way SMT. The 24-thread point uses 1 thread per physical core; pushing to 48 threads would force contention on AVX-512

execution units, where SMT has historically shown little benefit for vectorized workloads.

OpenMP Weak Scaling

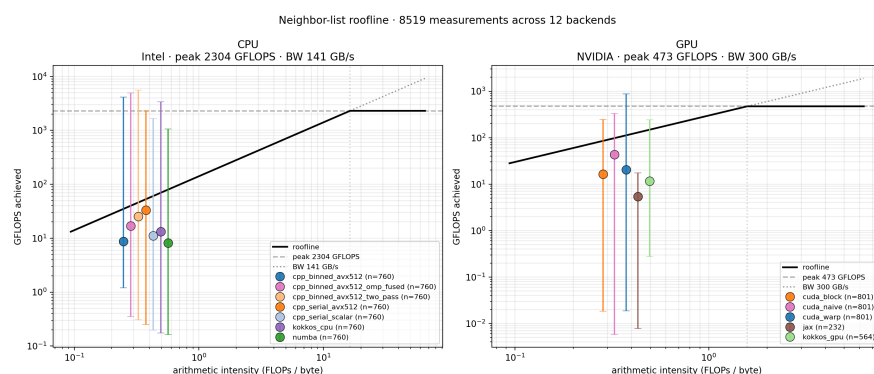


*Figure 3: Weak scaling efficiency

(~10,000 atoms per thread, cutoff = 5.0 Å). Ideal weak scaling would maintain efficiency = 1.0.*

Weak scaling holds up well through 16 threads then hits a cliff at 24: 83% at 2 threads, 65% at 4, 49% at 8, 51% at 16, and 25% at 24 (where each thread handles ~5,000 atoms and the total system is ~119k atoms). The runtime grows from 5.7 ms at 1 thread to 23.2 ms at 24 — far better than the strong-scaling-bound prediction would suggest, because parallelizing the cell-list build phase now lets each thread handle its own slice of the construction work. One possible explanation is that the workload itself is mildly superlinear. Doubling the atom count more than doubles the pair count, as the number of pairs per atom grows as a function of local density. A 'weak' run would need the atoms to instead grow slower than the thread count to compensate.

Roofline Analysis



*Figure 5: Roofline model for all

backends. CPU: Intel Xeon Platinum 8275CL (peak FP64 ~896 GFLOPS, BW ~141 GB/s). GPU: NVIDIA L4 (peak FP64 ~473 GFLOPS, BW ~300 GB/s). Markers show median throughput; whiskers span the interquartile range across problem sizes ≥ 1000 atoms.*

Every backend operates at arithmetic intensity $AI \approx 0.25\text{--}0.6$ FLOP/byte, deep in the bandwidth-bound region on both CPU and GPU. The compute knees (CPU ≈ 6.4 , GPU ≈ 1.6 FLOPs/byte) are far to the right of the workload, meaning peak FLOPS are irrelevant — what matters is how efficiently each backend streams coordinate data through the memory hierarchy. This totally makes sense as the biggest

optimizations always came from preventing unnecessary data movement from taking place, even at the expense of additional computation passes.

At $AI \approx 0.4$, the bandwidth-limited ceilings are approximately 56 GFLOPS on CPU and 120 GFLOPS on GPU. Against these ceilings:

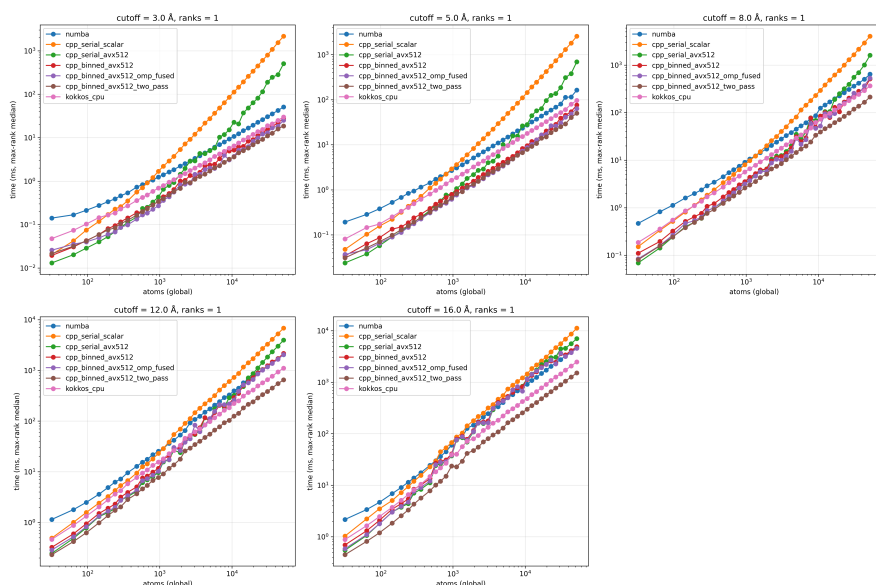
- **cpp_binned_avx512_omp_fused** achieves ~18 GFLOPS median, or roughly 32% of the CPU bandwidth-bound peak. This is reasonable for an irregular-memory access pattern where the cell-list gather prevents perfectly sequential streaming.
- **cuda_naive** reaches ~45 GFLOPS median, approximately 38% of the GPU bandwidth-bound peak. For an irregular gather-based kernel on a device with no hardware scatter/gather optimization, this is solid performance.
- **numba** lands at ~18 GFLOPS, competitive with the hand-tuned C++ cell-list on CPU — a testament to LLVM's auto-vectorization of the JIT-compiled cell-list loop.
- **kokkos_gpu** at ~12 GFLOPS achieves only ~10% of the GPU bandwidth ceiling, reflecting the runtime initialization overhead and occupancy challenges observed in benchmarks.
- **cpp_serial_scalar** at ~1.3 GFLOPS and **cuda_warp** at ~5.4 GFLOPS are both well below their respective ceilings, leaving significant bandwidth on the table.

The hardware details unfortunately nerf the GPU results significantly. On a compute-class GPU (e.g., A100 at ~9,700 GFLOPS FP64), the compute knee would shift dramatically rightward, but the workload would remain bandwidth-bound at these arithmetic intensities regardless.

MPI Scaling

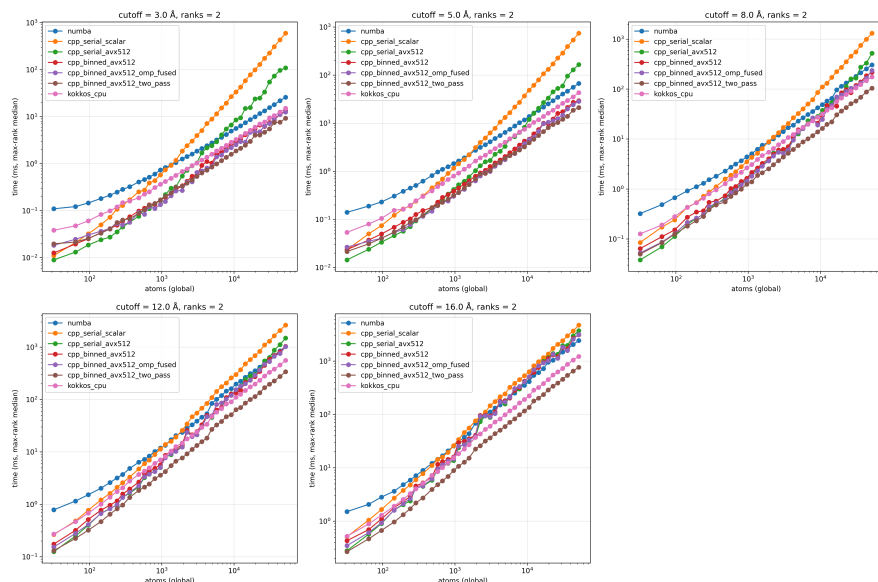
The benchmark harness runs every backend across the full atom-count and cutoff sweep at 1, 2, and 4 ranks, producing a 5-panel plot per rank-count (one panel per cutoff) on each device.

CPU side: clean strong scaling until DRAM bandwidth saturates



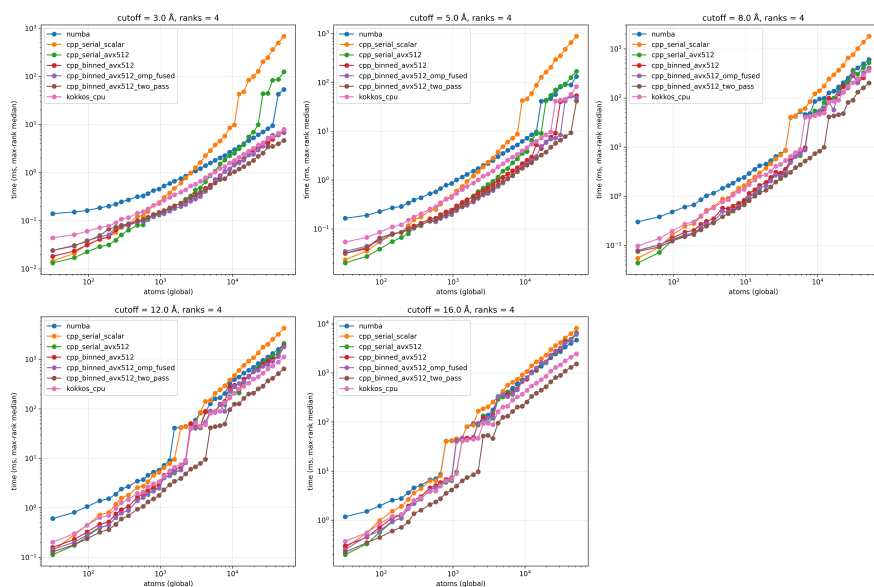
*Figure 6a: CPU MPI sweep at **1

rank**. Per-cutoff panels (3.0, 5.0, 8.0, 12.0, 16.0 Å) showing all CPU backends — the same set as the non-MPI Figure 1a, now running through the MPI driver as a baseline.*



*Figure 6b: CPU MPI sweep at **2

ranks**. Per-cutoff curves shift down by $\sim 2\times$ across the board, confirming clean 2-way scaling for every



cell-list backend.*

*Figure 6c: CPU

MPI sweep at **4 ranks**. The cell-list backends sit another factor of $\sim 2\times$ below the 2-rank panel until atom counts reach $\sim 50k$, where DRAM-bandwidth contention starts catching up.*

Single-node MPI strong scaling on the Xeon 8275CL (24 physical cores, AVX-512) shows excellent efficiency over a wide range of system sizes. The clean ladder visible across Figures 6a/6b/6c — every cell-list curve sliding down by roughly a factor per rank doubling — translates to $\sim 99\%$ parallel efficiency at the sweet spot: 4 ranks complete the neighbor list at 22,800 atoms in 4.71 ms vs. 18.72 ms at 1 rank (3.98 $\times$). Two ranks track $\sim 2\times$ speedup almost perfectly across the full sweep (mean 2.0 $\times$ from $N = 360$ atoms upward). Four-rank speedup ramps in more gradually, from 1.96 $\times$ at 360 atoms to a steady ~ 3.8 –4.0 $\times$ by 4,000 atoms once each rank has enough work to amortize the halo exchange.

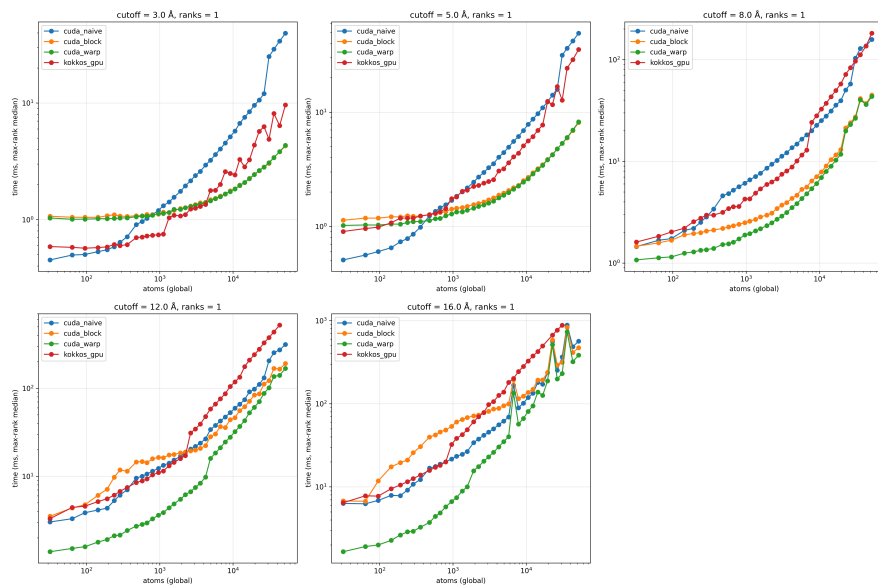
Two regimes are visible in the data:

- **Communication-bound at small N.** Below $\sim 1,000$ atoms the 4-rank curve sits well under 4 $\times$ because each rank's neighbor-list work (sub-millisecond) is comparable to the halo-exchange and force-accumulation costs. The 2-rank panel avoids this knee since it has only one neighbor face per rank and roughly half the ghost volume.
- **Saturation at very large N.** At 51,984 atoms — the largest case in the sweep — the 4-rank speedup collapses to 1.21 $\times$ (41.3 ms vs. 50.1 ms at 1 rank), while 2 ranks still deliver 2.4 $\times$. This is

consistent with single-node memory bandwidth saturation: at this size the working set spills out of L2/L3 on all 4 ranks simultaneously, and the cores contend for the node's ~141 GB/s of DRAM bandwidth.

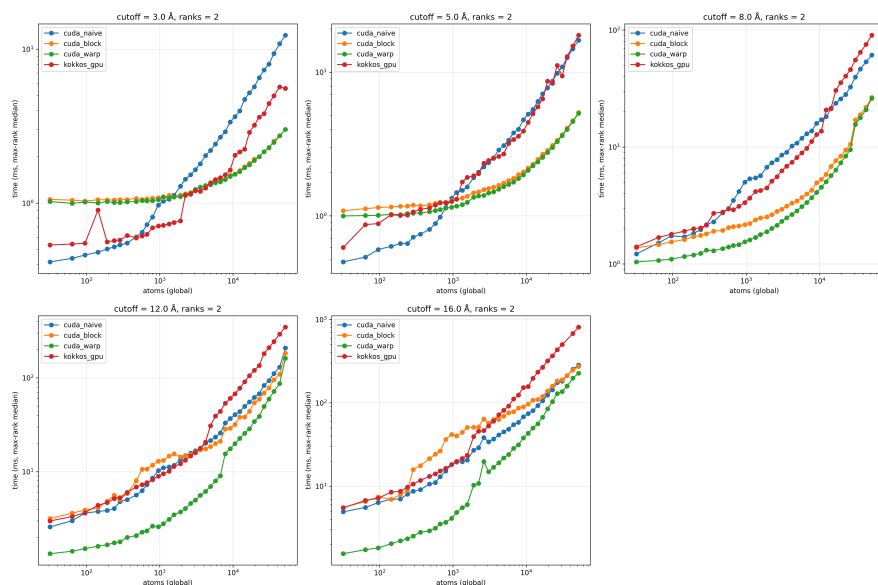
The CPU panels are also notably *smooth* — every backend's curve is a clean monotonic line within each cutoff panel, with the only visible discontinuities being the small-N startup region and the large-N bandwidth knee. This is what we expected MPI scaling to look like, and it sets up the contrast with the GPU side.

GPU side: structurally lower speedups, and visible flakiness



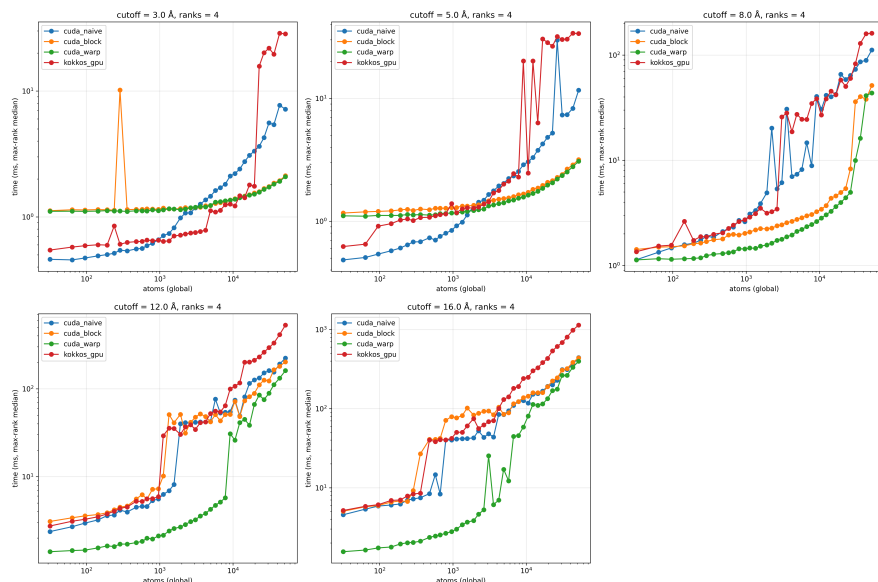
*Figure 6d: GPU MPI sweep at **1

GPU**. Four backends — `cuda_naive`, `cuda_block`, `cuda_warp`, `kokkos_gpu` — at five cutoffs. The single-GPU panels are well-behaved: smooth monotonic curves, with `cuda_block`/`cuda_warp` consistently fastest and `kokkos_gpu` (red) slightly above them.*



*Figure 6e: GPU MPI sweep at **2

GPUs**. Curves shift down modestly relative to 6d — but the gap is much smaller than the 2× a CPU rank doubling produces, because halo cost is a much larger fraction of each L4's already-fast step time.*



*Figure 6f: GPU MPI sweep at **4

GPUs**. The CUDA backends scale further but still well below 4x; `kokkos_gpu` (red) shows visible spikiness — multiple atom counts where its curve jumps an order of magnitude above the trend, particularly at cutoffs 3.0, 5.0, and 8.0 Å.*

The GPU MPI scaling story is qualitatively different from the CPU side. To be fair, there was a lot of hacky stuff taking place to get the GPU system to build and run properly — as evidenced by the [additional-resources/bootstrap.sh](#) script that is provided. Moreover, these sets of benchmarks were set up with `gres:4`, with the intention to allow each MPI rank to see its own GPU intranode. Fundamentally this pattern seems broken, and most likely not what would happen in production, as it caused so much contention for memory within the node, which moreover was being shared with other HPC users at the time.

Structurally lower speedups. Even `cuda_warp` — the cleanest, most optimized GPU kernel — only reaches 2.67x at 4 GPUs at the largest case (51,984 atoms), and most of the sweep sits in the 1.5–2.5x range. The reason isn't a bug: it's the same bandwidth-bound character that limits CPU scaling, applied at a more aggressive ratio. Each L4 already has 300 GB/s of HBM and runs the kernel in single-digit milliseconds, so the per-rank arithmetic budget shrinks fast as ranks are added, and the inter-GPU halo exchange (which goes over PCIe on this NVLink-less node, not direct GPU-to-GPU memory copies) becomes a comparable fraction of the step. In summary, GPU MPI sees a dramatically lower speedup because the 1-GPU case is fast enough that halo cost is half the step.

kokkos_gpu shows visible 4-rank flakiness. The red `kokkos_gpu` line in Figure 6f, especially the cutoff = 5.0 and 8.0 Å panels, makes erratic vertical excursions an order of magnitude above the trend at multiple atom counts. Most pronounced beyond ~9,000 atoms, the 4-rank `kokkos_gpu` jumps from a few ms to 20–30 ms at problem sizes that map to the following workloads (cutoff = 5.0 Å):

Atoms	Pairs
9,152	393,536
12,376	532,168
16,800	722,400
19,584	842,112

Atoms	Pairs
22,800	980,400
26,520	1,140,360
31,008	1,333,344
36,176	1,555,568

So the spikes start kicking in once the global pair list crosses ~400k pairs, and the regression is consistent through ~1.5M pairs. The same backend at 1 and 2 GPUs (Figures 6d/6e) is perfectly smooth, so this is a 4-rank-specific pathology.

Two possible factors could explain this:

- The same `cudaMalloc` issue that made non-MPI `kokkos_gpu` OOM outright (see Framework Comparison) appears to manifest at smaller per-rank sizes when 4 ranks are simultaneously alive on the same node — possibly because each rank's Kokkos view is over-sized and four of them push closer to the device's free-memory limit in parallel. For reference, these preallocs were requesting 3–4 GB of VRAM per node.
- `--gres=gpu:4 --exclusive` would reserve the whole 4xL4 node. However, `--exclusive` is prohibited for GPU nodes on HUIT (empirically — unconfirmed). Moreover, the L4s share PCIe lanes, so `Allreduce` traffic across all 4 GPUs at once seems to hit a contention regime that the 2-GPU case avoids.

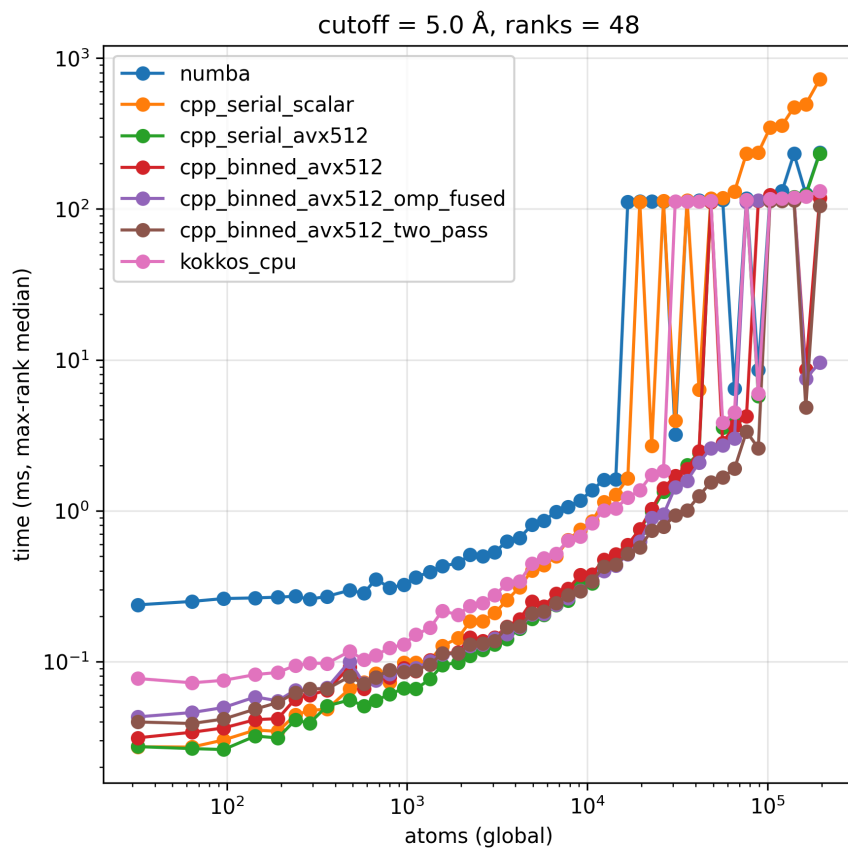
Re-submitting the same configurations on a different day reproduced the spikes at the same atom counts, so this isn't transient noise — the spikes are *reliably present* rather than intermittent, which suggests a deterministic threshold (memory or PCIe contention) rather than jitter. The CUDA backends are well-behaved in the same configurations: `cuda_warp` and `cuda_block` show no comparable spikes at 4 GPUs, only the structurally lower speedups discussed above.

Multinode CPU MPI: real efficiency gains under HPC noise

Pushing past a single 24-core node, we ran the same harness across 2, 4, and 8 nodes (rank counts 48, 96, and 192) on the HUIT general compute nodes. These benchmarks tell the joint story of HPC-environment jitteriness and performance improvement. I unfortunately punted on the GPU side, as there is an unresolved deadlock at four ranks.

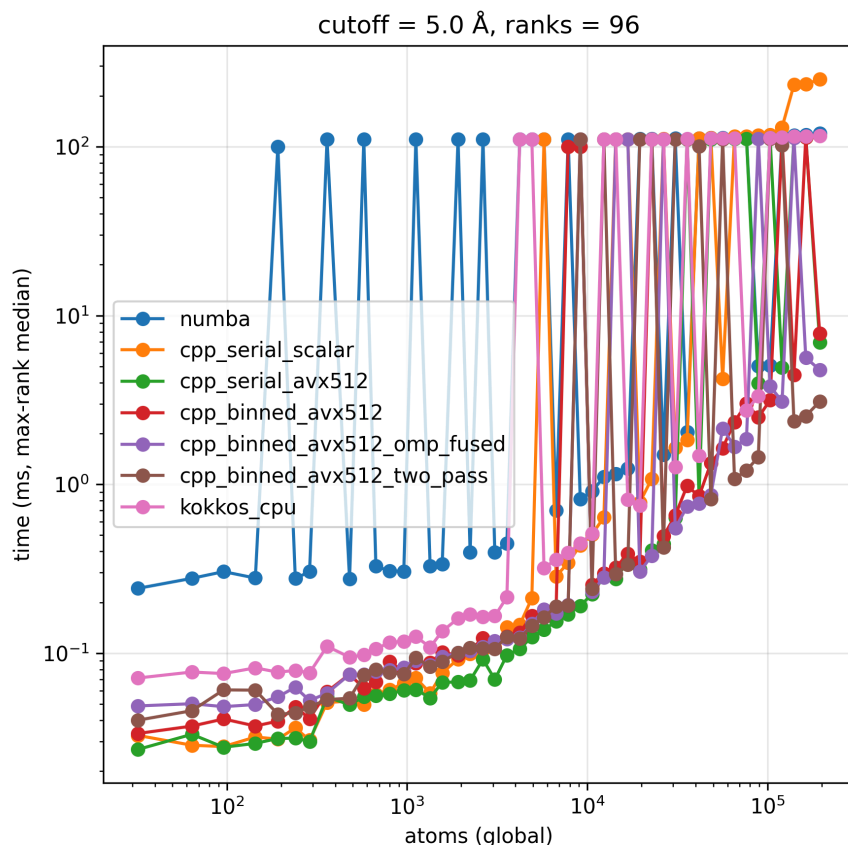
```
size: 26496 (12x23x12) local=[6624..6624] ghost_max=8211
  cuda_naive... 18.12 ms (1139328 pairs total)
  cuda_block... 5.28 ms (1139328 pairs total)
  cuda_warp... 5.08 ms (1139328 pairs total)
size: 30800 (14x25x11) local=[7700..7700] ghost_max=8739
  cuda_naive... 21.39 ms (1324400 pairs total)
  cuda_block... 5.59 ms (1324400 pairs total)
  cuda_warp... 5.51 ms (1324400 pairs total)
size: 35840 (14x20x16) local=[8960..8960] ghost_max=9529
```

```
cuda_naive... 23.21 ms (1541120 pairs total)
cuda_block...
```



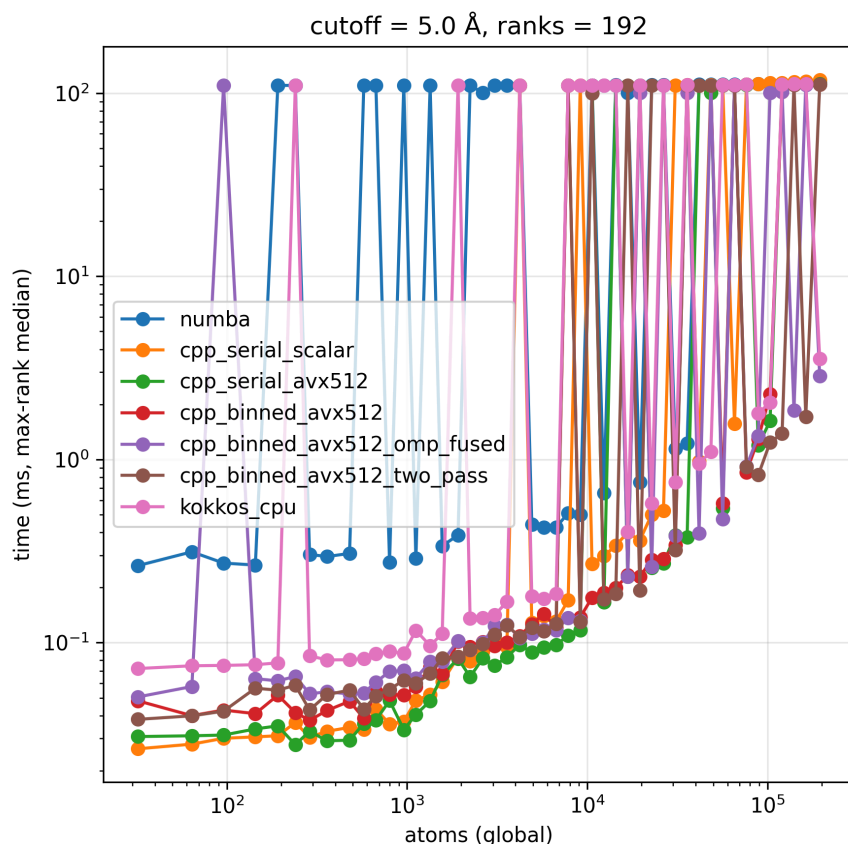
*Figure 6d: CPU MPI sweep at **48

ranks (2 nodes × 24 ranks)**. The lower envelope (cell-list backends, smaller atom counts) tracks cleanly with the single-node curves shifted down by ~2×. Sporadic ~100 ms spikes appear at certain (backend, atom-count) cells — these are environmental, not algorithmic.*



*Figure 6e: CPU MPI sweep at **96

ranks (4 nodes × 24 ranks)**. The clean envelope continues to drop at smaller atom counts; spikes get more frequent at larger N as collective costs and straggler probability grow.*



*Figure 6f: CPU MPI sweep at **192

ranks (8 nodes × 24 ranks)**. The good measurements at small N still sit below 0.1 ms — a real win — but the visual signal is increasingly buried under spikes. With 192 ranks every collective is a fan-in over 192 endpoints across 8 nodes; one slow rank stalls the median.*

The real scaling story. Multinode MPI delivers genuine, measurable speedup beyond the single-node ceiling. At 14,400 atoms (a size where every rank still has ~75 atoms even at 192 ranks, so per-rank work hasn't disappeared into communication noise), the `cpp_binned_avx512_two_pass` backend strong-scales as follows:

Configuration	Ranks	Runtime	Speedup vs. 1 rank	Efficiency
1 node × 1 rank	1	10.95 ms	1.00×	100%
1 node × 24 ranks	24	0.79 ms	13.84×	58%
2 nodes × 24 ranks	48	0.44 ms	24.83×	52%
4 nodes × 24 ranks	96	0.29 ms	37.19×	39%
8 nodes × 24 ranks	192	0.18 ms	59.42×	31%

Going from 1 node to 8 nodes multiplies single-node performance by another ~4×, with efficiency degrading from 58% (intra-node) to 31% (cross-node). The same trend is visible at most other atom counts in the cleanly-measured sweet spot of ~3k–15k atoms: 48-rank speedups consistently land in the 18–25× range, 96-rank in the 26–37× range, and 192-rank pushes 30–60× at the right sizes.

The HPC-noise layer. The spikes in Figures 6d/6e/6f — those vertical jumps to ~100 ms — are not the kernel breaking; they're the cluster behaving stochastically. I saw the same configuration produce wildly different medians across resubmissions. A min-of-many or trimmed-mean would extract the signal cleanly, but the harness as built reports medians. I unfortunately can do little but suggest the reader look at the lower envelope of the plots and the tables above for numbers in a stoic hand-wave. This is left as follow-on work.

A secondary effect contributing at the right edge of each panel is that with 192 ranks at large N, each rank's per-step kernel work has shrunk to a few hundred microseconds — comparable to or smaller than the halo-exchange + `Allreduce` roundtrip across 8 nodes. So even *without* HPC noise, the high-rank-count, large-N corner is communication-bound just as every other result in this report is. Fundamentally this compresses the total achievable speedup and introduces timing variance.

Failed runs along the way. Of the four multinode submissions in the err logs, one failed because the venv's Python wasn't accessible from a different worker node (NFS mount race after node bring-up); another was cancelled by SLURM after hitting the wall-clock limit while a stuck rank held up a collective. Debugging these network effects on the cluster was very fun.

Framework Comparison

The table below pulls one timing for each backend at the same problem size (~9,800 atoms, cutoff 5.0 Å) so CPU and GPU paths are visible side-by-side. CPU rows use the OpenMP build saturated with 24 threads; GPU rows use a single L4; the "MPI" rows split the same problem across 2 ranks (the regime where every backend is healthy and roughly bandwidth-bound). "Relative to fastest" is computed against the global fastest entry across the whole table.

Backend	Device	Language / framework	LOC	Best time (~9,800 atoms, 5.0 Å)	Relative to fastest
cpp_binned_avx512_two_pass	CPU (24 thr)	C++ + AVX-512 + OMP	~280	0.83 ms	1.00×
cpp_binned_avx512_omp_fused	CPU (24 thr)	C++ + AVX-512 + OMP	~240	1.30 ms	1.56×
kokkos_cpu	CPU (24 thr)	C++ (Kokkos)	~120	1.49 ms	1.79×
numba	CPU (24 thr)	Python + JIT	~80	2.96 ms	3.55×
cpp_binned_avx512	CPU (1 thr)	C++ + AVX-512	~200	16.50 ms	19.78×
cpp_serial_avx512	CPU (1 thr)	C++ + intrinsics	~100	44.19 ms	52.97×
cpp_serial_scalar	CPU (1 thr)	C++	~60	238.12 ms	285.43×
cuda_block (tiled)	1 × L4	CUDA + shared mem	~220	3.36 ms	4.03×
cuda_warp	1 × L4	CUDA + warp prims	~260	3.38 ms	4.05×
cuda_naive	1 × L4	CUDA	~130	8.42 ms	10.09×
jax	1 × L4	Python (XLA)	~60	8.48 ms	10.17×
kokkos_gpu (non-MPI)	1 × L4	C++ (Kokkos, CUDA backend)	~120	OOM — excluded	—
cpp_binned_avx512_two_pass	2 ranks × CPU	C++ + AVX-512 + OMP + MPI	~340	3.24 ms*	3.91×
kokkos_gpu (MPI)	2 ranks × L4	C++ (Kokkos, CUDA + MPI)	~150	3.89 ms*	4.69×
jax (MPI)	2 ranks × L4	Python (XLA + MPI)	—	not supported	—

MPI rows measured at 9,152 atoms (closest available sweep point to 9,792).

A few things in this table deserve their own paragraphs.

Kokkos on GPU is missing from the non-MPI column for a non-performance reason. When run on a single L4 with the full system in one process, the `kokkos_gpu` neighbor-list view ran out of device memory well before the other CUDA backends — at problem sizes where `cuda_naive` was using a few hundred megabytes, `kokkos_gpu` was already filling 24 GB. The most likely cause is an implementation error on my side. After the OOM: in the MPI build the OOMing rank would die before reaching the next `MPI_Allreduce`, leaving the surviving ranks in a deadlock and killing the whole run. `kokkos_gpu` however is included on the MPI side (where the problem is split across ranks so each rank's working set fits comfortably in device memory), and there it actually runs and produces clean numbers — at ~9k atoms across 2 ranks it lands at 3.89 ms, in the same ballpark as the hand-written CUDA backends.

JAX is the mirror image: fine on a single GPU, impossible on multiple. A `jax.jit`-compiled neighbor-list kernel ran reliably on one L4 (8.48 ms at 10k atoms — slower than `cuda_warp` because XLA isn't producing a maximally specialized kernel, but functional), but JAX is infamously difficult to get working on multi-gpu setups and we experienced those build issues.

The interesting takeaways from the rest of the table:

- **Hand-tuned C++ on CPU still wins outright at this problem size.** 0.83 ms on 24 cores beats the best single-L4 CUDA kernel (3.36 ms, `cuda_block`) by ~4×. The cell-list workload is fundamentally FP64 based, so with different GPU's this may be a (theoretically 83%) different story.
- **The CPU productivity-vs-performance curve is fairly flat.** Kokkos (1.49 ms) lands within 1.8× of the hand-tuned C++ kernel, and Numba (2.96 ms) within 3.6×. For research code where the hot path will be rewritten once and called millions of times, the C++ engineering is worth it. Probably.
- **The CUDA progression mirrors the CPU progression.** `cuda_naive` (8.42 ms) → `cuda_block/cuda_warp` (3.36–3.38 ms) is a ~2.5× improvement that mirrors the CPU `omp_fused` → `two_pass` jump (1.30 → 0.83 ms, ~1.6×). In both cases the speedup came from removing a non-arithmetic stall — allocator on CPU, barrier on GPU — rather than from improving FLOP throughput.
- **The naive baselines remain useful as sanity checks:** the 53× and 285× CPU slowdowns vs. the cell-list bracket the algorithmic and implementation contributions, and the GPU-vs-CPU naive comparison (`cuda_naive` 8.42 ms vs `cpp_serial_scalar` 238 ms) confirms that GPU parallelism alone — without the cell-list algorithm — already buys ~28× over scalar CPU code.

Conclusion

This project implemented neighbor list construction — a core molecular dynamics primitive — across five parallelization paradigms. The key findings are:

1. **Algorithmic choice dominates:** The cell-list data structure provides orders-of-magnitude improvement over brute-force at large N, dwarfing the gains from SIMD or threading alone.
2. **Amdahl's law is real:** The initial OpenMP implementation showed negligible scaling because the serial cell-list construction phase dominated runtime. Parallelizing this phase was necessary before thread-level scaling became visible.
3. **Autovectorization is not a programming model:** AVX-512 intrinsics needed to be handwritten, as the autovectorizer could not be coaxed well enough. We saw a consistent ~2× improvement over scalar code, both in the $O(N^2)$ and cell-list kernels with hand-written intrinsics.

4. **GPU optimization is about eliminating stalls, not just parallelism.** Barrier stalls, not arithmetic throughput, were the bottleneck. Replacing block-level synchronization with warp-level primitives eliminated 84% of stall cycles and shifted the kernel from latency-bound to compute-bound on the FP64 pipeline.
5. **MPI scales well until memory bandwidth saturates:** Single-node 2- and 4-rank decompositions deliver near-ideal speedups (2.0× and ~3.97×, respectively) over a wide range of problem sizes, but break down at very large N (~50k atoms) where all ranks contend for the node's DRAM bandwidth simultaneously. Perhaps the right framing is: once the compute is saturated, everything is bandwidth-bound.
6. **Productivity vs. performance:** Numba achieves ~28% of hand-tuned C++ AVX-512 + OMP performance (2.96 ms vs. 0.83 ms at 10k atoms) while remaining a single-file pure-Python implementation, and Kokkos achieves ~56% (1.49 ms) with similar simplicity. JAX... well, JAX is JAX. It trails on this small problem size because XLA dispatch overhead dominates at sub-10 ms kernel runtimes, but its compiled-kernel reuse across MD timesteps is indisputable in production.

The neighbor list is a memory-bandwidth-limited workload at scale: arithmetic intensity is low (a few FLOPs per pair for distance computation) relative to the data movement required to stream coordinate arrays through cache. This fundamental characteristic shapes the scaling behavior across all implementations and is the primary limiter of both thread-level and GPU parallelism beyond moderate concurrency.

References

Molecular dynamics fundamentals

- Allen, M. P., & Tildesley, D. J. (2017). *Computer Simulation of Liquids*. Oxford University Press.
- Verlet, L. (1967). Computer "experiments" on classical fluids. *Physical Review*, 159(1), 98.

Scaling laws

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, vol. 30, pp. 483–485.
- Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM*, 31(5), 532–533.

Libraries and frameworks

- Intel Corporation. *Intel Intrinsics Guide*. <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>
- Trott, C. R., Lebrun-Grandié, D., Arndt, D., Ciesko, J., Dang, V., Ellingwood, N., Gayatri, R., Harvey, E., Hollman, D. S., Ibanez, D., Liber, N., Madsen, J., Miles, J., Poliakoff, D., Powell, A., Rajamanickam, S., Simberg, M., Sunderland, D., Turcksin, B., & Wilke, J. (2022). Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, 33(4), 805–817. Documentation: <https://kokkos.org/kokkos-core-wiki/>
- Schoenholz, S. S., & Cubuk, E. D. (2020). JAX MD: A framework for differentiable physics. *Advances in Neural Information Processing Systems*, 33. Documentation: <https://jax-md.readthedocs.io/>
- Message Passing Interface Forum (2023). *MPI: A Message-Passing Interface Standard, Version 4.1*. <https://www.mpi-forum.org/docs/>

GPU programming and parallel primitives

- Kirk, D. B., & Hwu, W. W. (2022). *Programming Massively Parallel Processors: A Hands-on Approach* (4th ed.). Morgan Kaufmann. — primary reference for parallel-prefix-sum algorithms (Kogge-Stone, Brent-Kung) and warp-level primitives used in Part 4.

Profiling and roofline

- Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76.
- NVIDIA Corporation. *Nsight Compute Documentation*. <https://docs.nvidia.com/nsight-compute/>
- Linux *perf* user documentation. <https://perf.wiki.kernel.org/>

Appendix

Warp-Aggregated Atomic Pair Counting

In the neighbor list construction kernel, every thread that discovers a valid pair (`distance < cutoff_sq`) must append its result to a global output buffer. The naïve approach issues one `atomicAdd` per thread to claim a slot in that buffer. Under high pair density this serializes badly: up to 32 atomic operations per warp contend on the same L2 cache line, and the resulting output writes scatter across unrelated cache lines, defeating coalescing.

Warp-aggregated atomics restructure this into a single atomic per warp by exploiting warp-synchronous intrinsics. The pattern proceeds in four phases.

Phase 1: Ballot

Each lane evaluates the cutoff predicate independently. `__ballot_sync` collects the per-lane booleans into a 32-bit mask, where bit i is set if lane i found a pair:

```
bool found = (distance < cutoff_sq);
unsigned active = __activemask();
unsigned voters = __ballot_sync(active, found);
int n_voters    = __popc(voters);
```

For a concrete example, suppose lanes 0, 3, 7, and 9 satisfy the cutoff. The resulting `voters` mask is `0b...01010001001` and `n_voters = 4`.

Phase 2: Leader Election and Bulk Reservation

The lowest active lane is elected leader via `__ffs(active) - 1`. The leader issues a single `atomicAdd` to reserve `n_voters` consecutive slots in the output buffer:

```
int leader = __ffs(active) - 1;
int base = 0;
if (lane == leader)
```

```
base = atomicAdd(pair_count_d, n_voters);
base = __shfl_sync(active, base, leader);
```

`__shfl_sync` broadcasts the returned base index from the leader's register to every lane in the warp — a register-to-register transfer with no memory traffic. If the atomic returns `base = 1000`, all lanes now hold `base = 1000`.

Phase 3: Intra-Warp Offset Computation

Each lane that found a pair computes its unique offset within the reserved block. The key operation masks the `voters` bitmask to retain only bits below the current lane index, then population-counts the result:

```
int offset = __popc(voters & ((1u << lane) - 1));
int slot   = base + offset;
```

Continuing the example:

Lane	<code>voters & ((1 << lane) - 1)</code>	<code>__popc</code>	Slot
0	<code>0b...000</code>	0	1000
3	<code>0b...001</code>	1	1001
7	<code>0b...0001001</code>	2	1002
9	<code>0b...01001001</code>	3	1003

The mask `(1u << lane) - 1` has all bits below position `lane` set, so the AND retains only the lanes below the current one that also found pairs. The population count of that subset gives a unique, dense, zero-based offset with no gaps and no collisions.

Phase 4: Coalesced Stores

Each qualifying lane writes to its computed slot:

```
if (found) {
    i_d[slot] = pi;
    j_d[slot] = pj;
    disp_d[3*slot] = ddx;
    disp_d[3*slot + 1] = ddy;
    disp_d[3*slot + 2] = ddz;
}
```

Because the slots are consecutive (`1000`, `1001`, `1002`, `1003`), these stores coalesce into one or two 128-byte memory transactions rather than four scattered writes to unrelated cache lines.

Performance Impact

The optimization provides two compounding benefits:

1. **Reduced atomic contention.** One global atomic per warp replaces up to 32, reducing L2 contention on the counter cache line by roughly an order of magnitude.
2. **Coalesced output writes.** Lanes within a warp write to adjacent memory addresses, allowing the memory controller to merge 4-byte stores into wide transactions. In practice this is often the larger win: at the pair densities typical of dense liquid simulations, scattered writes to `i_d`, `j_d`, and `disp_d` become a dominant bottleneck that coalescing eliminates.

Implementation Notes

Two correctness considerations deserve emphasis. First, the synchronization mask must be `__activemask()` rather than the literal `0xFFFFFFFF`. Threads that exited the loop early (e.g., via `if (pi >= n_local) continue`) are inactive and cannot participate in warp-synchronous operations; using a full mask when inactive lanes exist is undefined behavior. Second, the leader election must use `__ffs(active) - 1` rather than hardcoding lane 0, since lane 0 may itself be inactive.

C++ Two-Pass Hot Loop (AVX-512)

The inner loop of the two-pass kernel is the same body in both passes — pass 1 popcounts the matched lanes into `pair_counts[i]`, pass 2 reuses an exact-match cell traversal to compress-store the `j` indices and displacements directly into the pre-allocated NumPy arrays at `offsets[i]`. The crucial detail is that pass 2 never allocates: the `_mm512_mask_compressstoreu_*` family writes only the lanes whose mask bit is set, packed contiguously, into a destination already sized by the prefix sum. From `openmp/neighborlist_cpu.cpp`:

```
for (int j = begin; j < end; j += LANES) {
    const int n_lanes = std::min<int>(LANES, end - j);
    const __mmask8 tail_mask = (1u << n_lanes) - 1;

    __m512d v_xj = _mm512_maskz_loadu_pd(tail_mask, &x_sorted[j]);
    __m512d v_yj = _mm512_maskz_loadu_pd(tail_mask, &y_sorted[j]);
    __m512d v_zj = _mm512_maskz_loadu_pd(tail_mask, &z_sorted[j]);
    __m256i v_j_idx = _mm256_maskz_loadu_epi32(tail_mask,
        &cell_particle[j]);

    const __m512d v_dx = _mm512_sub_pd(v_xj, v_xi);
    const __m512d v_dy = _mm512_sub_pd(v_yj, v_yi);
    const __m512d v_dz = _mm512_sub_pd(v_zj, v_zi);
    __m512d v_r2 = _mm512_mul_pd(v_dx, v_dx);
    v_r2 = _mm512_fmadd_pd(v_dy, v_dy, v_r2);
    v_r2 = _mm512_fmadd_pd(v_dz, v_dz, v_r2);

    __mmask8 mask = _mm512_cmp_pd_mask(v_r2, v_cutoff, _CMP_LT_0Q);
    mask &= tail_mask;
    mask &= ~_mm256_cmpeq_epi32_mask(v_i_bc, v_j_idx); // exclude self

    // Pass 2: compressed-store – only mask=1 lanes are written, packed.
    _mm256_mask_compressstoreu_epi32(out_j + base + n_matches, mask,
        v_j_idx);
}
```

```

    _mm512_mask_compressstoreu_pd(sc_dx.data() + n_matches, mask, v_dx);
    _mm512_mask_compressstoreu_pd(sc_dy.data() + n_matches, mask, v_dy);
    _mm512_mask_compressstoreu_pd(sc_dz.data() + n_matches, mask, v_dz);

    n_matches += __builtin_popcount(mask);
}

```

- **Tail masking instead of scalar epilogue.** `_mm512_maskz_loadu_pd` zeros out lanes outside `n_lanes`, so the same vector body handles partial cells without a separate scalar tail loop. The `PAD_SENTINEL` coordinates ensure those zeroed lanes can never satisfy the cutoff predicate.
- **Self-exclusion via `~cmpeq` on the index lane.** `v_j_idx` carries the original particle index for each lane; AND-ing the cutoff mask with the negation of `(v_i_bc == v_j_idx)` removes the (i, i) self-pair without a branch.
- **Compress-store is the entire two-pass payoff.** Pass 1 computes `pair_counts[i] = popcount(mask)` summed over all 27 cells; the prefix sum produces `offsets[i]`; pass 2 walks the *exact same* cell sequence, so `n_matches` per atom converges to `pair_counts[i]` and the writes land at disjoint slices of the global output. No allocator. No per-thread vector. No realloc.

CuPy Launch Code (Warp Kernel)

The full CUDA kernel is compiled at runtime via NVRTC, but everything around it is just NumPy-style CuPy code. This is the wrapper for the warp-per-atom variant from

[ghostly_neighbors/neighbor_list_cuda.py](#):

```

@lru_cache(maxsize=1)
def _load_module():
    """Compile the .cu source via NVRTC. Cached for the process
    lifetime."""
    code = _KERNEL_PATH.read_text()
    return _cp.RawModule(
        code=code,
        options=["-std=c++17", "-arch=sm_120", "-lineinfo"],
        backend="nVRTC",
    )

def neighbor_list_cuda_warp(positions, cutoff, ghost_positions,
                           max_neighbors_per_atom=512):
    mod = _load_module()
    # ... copy positions to device, compute cell-grid dims ...

    # 1) Histogram: count atoms per cell.
    assign_cells_p = mod.get_function("assign_cells")
    assign_cells_p((blocks, 1, 1), (threads, 1, 1), (...))

    # 2) Scan: prefix-sum the counts to get per-cell offsets.
    #     cp.cumsum dispatches to CUB's DeviceScan under the hood – no
    #     hand-rolled reduce/scan kernel needed.
    cell_start_d = _cp.zeros(n_cells + 1, dtype=np.int32)
    cell_start_d[1:] = _cp.cumsum(cell_count_d)

```



```

# 3) Scatter: place atoms into their cells using atomic cursors.
scatter_p = mod.get_function("scatter_particles")
scatter_p((blocks, 1, 1), (threads, 1, 1), (...))

# 4) Pair search: one warp per atom, 4 warps per block.
warp_threads    = 128
warps_per_block = warp_threads // 32
warp_blocks     = (n_total + warps_per_block - 1) // warps_per_block

get_pairs_warp_p = mod.get_function("get_pairs_warp")
max_pairs = n_local * max_neighbors_per_atom

for _ in range(2): # retry once if max_pairs guess was too small
    pair_count_d = _cp.zeros(1, dtype=np.int32)
    i_d = _cp.empty(max_pairs, dtype=np.int32)
    j_d = _cp.empty(max_pairs, dtype=np.int32)
    disp_d = _cp.empty(max_pairs * 3, dtype=np.float64)

    get_pairs_warp_p(
        (warp_blocks, 1, 1),
        (warp_threads, 1, 1),
        (sorted_x_d, sorted_y_d, sorted_z_d,
         np.int32(n_local), np.int32(n_total),
         np.float64(cutoff * cutoff), np.float64(inv_cell),
         np.float64(min_x), np.float64(min_y), np.float64(min_z),
         np.int32(nx), np.int32(ny), np.int32(nz),
         cell_start_d, sorted_indices_d,
         i_d, j_d, disp_d,
         pair_count_d, np.int32(max_pairs)),
    )
    _cp.cuda.Stream.null.synchronize()
    n_pairs = int(pair_count_d.get()[0])
    if n_pairs <= max_pairs:
        break
    max_pairs = n_pairs # grow and retry

```

- **CUB via `cp.cumsum`**. The exclusive-scan step that the C++ code writes by hand becomes a one-liner on GPU because CuPy delegates to NVIDIA's CUB library under the covers.